

Introduction

In this codelab, you will learn how to set up your GitHub account for contributing to [BRAND NAME] repositories.

Prerequisites

Before you begin this codelab, you will need:

- An [BRAND NAME] email address
- An invitation to join the GitHub organization

NOTE: If you do not have an invitation to join the [BRAND NAME] GitHub organization, reach out to your manager to obtain one. For more information, see the article [How to send an invitation to the \[BRAND NAME\] GitHub Organization? in AnswerHub](#).

Additionally, in order to complete this and other related codelabs, you need to have the following software installed locally:

- A Linux-based operating system:
 - GNU/Linux
 - Windows Subsystem for Linux 2 (WSL2)
 - macOS
- [git](#)
- bash or zsh shell

What you'll accomplish

By the end of this codelab, you will have successfully:

- Joined the [BRAND NAME] GitHub organization
- Configured your GitHub user
- Tested your configuration on an existing [BRAND NAME] repository

SECTION 1

Get started

You can join the [BRAND NAME] GitHub organization by creating a new account with your [BRAND NAME] email or, if you already have a GitHub account, by linking your [BRAND NAME] email to your existing GitHub account.

Join by creating a new GitHub account

To create a new GitHub account with your [BRAND NAME] email:

1. Go to [GitHub](#).
2. Click the **Sign up** button on the top-right corner.

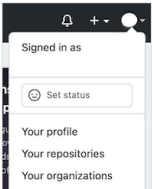


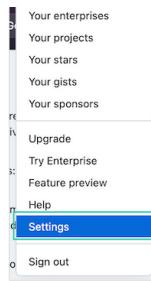
3. Enter your [BRAND NAME] email and follow the process to complete the registration.
4. [Verify your email address](#).

Join by linking your [BRAND NAME] email to an existing account

To join by linking your [BRAND NAME] email to an existing GitHub account:

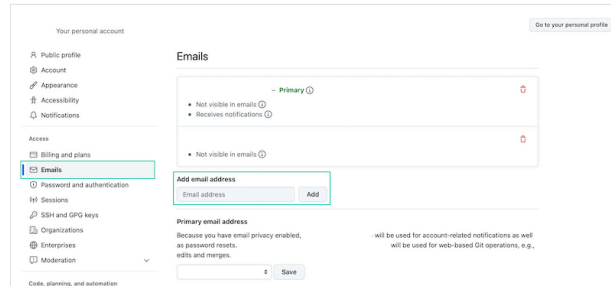
1. Go to [GitHub](#).
2. Log in with your existing credentials.
3. Click on your profile picture and go to **Settings**.





4. In the "Access" section of the sidebar, click on **Emails**.

5. In "Add email address" field, type your [BRAND NAME] email address and click on the **Add** button.



6. Verify your email address.

7. In the "Primary email address" list, you can set up your [BRAND NAME] email address as the account to use as your main web-based Git operations account.

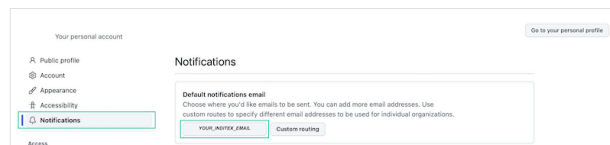
SECTION 2

Configure your GitHub account

Once you have joined the [BRAND NAME] GitHub organization, you need to configure the following settings:

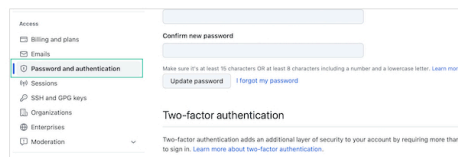
1. Configure your notifications:

1. Go to **Settings** from your [GitHub](#) account.
2. Click on the **Notifications** option from the left sidebar.
3. Select your [BRAND NAME] email to receive notifications.

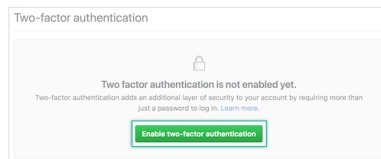


2. Enable **two-factor authentication (2FA)**.

1. Go to **Settings** from your [GitHub](#) account.
2. Navigate to the **Password and authentication** tab from the left panel.

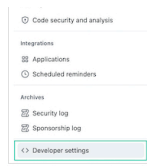


3. Go to the **Two-factor authentication** section and click on the **Enable two-factor authentication** button.



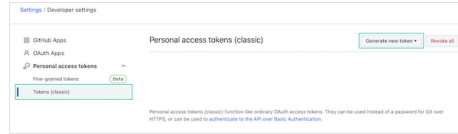
4. Open the **RSA SecurID Authenticate** application on your smartphone and scan the QR code.
The application saves your GitHub account and generates a new authentication code every few seconds.
 5. Type the code to verify the verification field on GitHub.
 6. Save your recovery codes and complete the registration process.
3. Follow the guide to [create a Personal Access Token \(PAT\)](#).
1. Go to **Settings** from your [GitHub](#) account.
 2. Navigate to the **Developer settings** tab from the left panel.





3. Navigate to the **Personal access tokens** tab from the left panel and select the **Tokens (classic)** option.

4. Click **Generate new token**.



5. Enter a name for your token and select an expiration period.

6. Select the applicable scopes for the PAT.

7. Click **Generate token**.

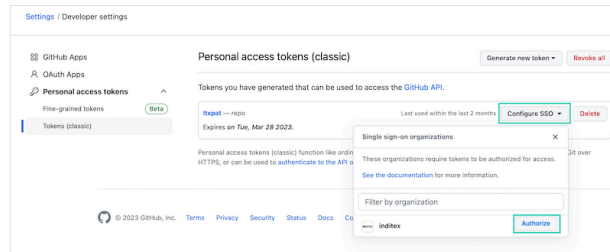
4. Authorize your PAT to authenticate to the [BRAND NAME] organization using **SAML SSO**.

1. Go to **Settings** from your **GitHub** account.

2. Navigate to the **Developer settings** tab from the left panel.

3. Navigate to the **Personal access tokens** tab from the left panel and select the **Tokens (classic)** option.

4. Click the **Configure SSO** button next to your new token and click the **Authorize** button to authorize your token for the [BRAND NAME] organization.



5. Follow the guide to **generate a new GPG key** using your [BRAND NAME] email details.

1. From a Terminal window, run the following command to generate a GPG key pair:

```
gpg --full-generate-key
```

2. Press **Enter** to set the default values for your key through the creation process.

3. Enter your user ID information (remember to use your [BRAND NAME] email address and make sure your email is **verified**).

4. Type a secure passphrase.

5. Run the following command to list the long form of the GPG key:

```
gpg --list-secret-keys --keyid-format=long
```

6. Copy the long form of the GPG key ID. In the following example, the GPG key ID is **3AA5C34371567BD2**:

```
$ gpg --list-secret-keys --keyid-format=long
/Users/hubot/.gnupg/secring.gpg
-----
sec  4096R/3AA5C34371567BD2 2016-03-10 [expires: 2017-03-10]
uid                  Hubot <hubot@example.com>
ssb  4096R/4B86D45482678BE3 2016-03-10
```

7. Run the following command, substituting the GPG key ID for your own to print the GPG key ID in ASCII armor format:

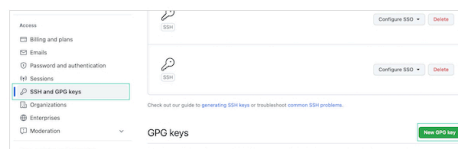
```
gpg --armor --export 3AA5C34371567BD2
```

8. Copy your GPG key, beginning with **-----BEGIN PGP PUBLIC KEY BLOCK-----** and ending with **-----END PGP PUBLIC KEY BLOCK-----**.

9. Go to **Settings** from your **GitHub** account.

10. Navigate to the **SSH and GPG keys** tab from the left panel.

11. Click **New GPG key**.





12. In the "Key" field, paste the GPG key you copied previously (ASCII armor format).

13. Click the **Add GPG key** button.

6. Setup the GitHub CLI

1. Install the [GitHub CLI](#)

2. To log into the GitHub CLI, enter the following command:

```
gh auth login
```

```
# Select: GitHub.com > HTTPS > y > Login with a web browser
```

NOTE: This returns a one-time code.

3. Copy the one-time code

4. Press the **Enter** key to open GitHub.com

5. Paste the one-time code and **Continue**

6. Click on **Authorize GitHub**

7. Enter your GitHub two-factor authentication and **Verify**

Set up the Sherpa GitHub command extension

With your GitHub account set up, you need to install the [Sherpa](#) GitHub command extension for your branching and pull request operations.

Enter the following command:

```
gh extension install inditex/gh-sherpa
```

NOTE: You might need to run `gh auth login` first and log into your GitHub account if you haven't used the GitHub CLI before.

SECTION 3

Test your configuration

Follow the next steps to test your configuration and make sure you have set up your local environment correctly. You will clone an existing repository, make a contribution to it, and create a pull request to merge your changes.

Clone the sample repository

For this code lab, you will clone a sample sandbox project and will create a pull request to verify your configuration is working as expected.

1. Clone the following repository:

```
git clone https://github.com/inditex/mic-localcfgcode lab.git
# Username: {GitHub username}
# Password: {PAT}
```

2. Change directory:

```
cd mic-localcfgcode lab/
```

3. Create an issue:

```
gh issue create
# Type: "YOUR_NAME_contribution" as the issue Title > Enter > Enter
```

❶ Replace `YOUR_NAME` with your name for your issue to be unique for this exercise.

NOTE: This returns the URL of your GitHub issue. Copy the issue ID from the URL by taking the number located at the end of the URL.

4. Create a branch:

```
gh sherpa create-branch --issue ❶ISSUE_ID
```

❶ Replace `ISSUE_ID` with the ID you copied from the URL.

1. When prompted, select `documentation` as branch type and press `Enter`.

2. Enter a description (for example: "Test contribution") and press Enter.
 3. Type "Y" to confirm the branch creation and press Enter.
- You receive a confirmation message that the branch has been created.

Configure git to sign your commits and use your [BRAND NAME] email address

To configure git to sign your commits and use your [BRAND NAME] email address in a repository:

1. In your active repository, run the following command:

```
git config commit.gpgsign true
```

2. Run the following command to set your GPG signing key:

```
git config user.signingkey YOUR_GPG_KEY_ID
```

NOTE: To get your GPG key ID, go to GitHub > Settings > SSH and GPG keys and navigate to the GPG Keys section. The information card of your active GPG key contains the details of your GPG key ID.

3. Run the following commands to set your git name and email:

```
git config user.name "YOUR NAME"  
git config user.email YOUR_@_ADDRESS_.com
```

NOTE: You can configure the previous settings globally by adding the `--global` flag in the `config` commands. For example: `git config --global commit.gpgsign true`. For more details, see the [Sign commits with GPG keys](#) page in the GitHub Ecosystem Docs.

Add your changes and create a pull request

Finally, it's time to add your changes to the repository and create a pull request.

1. Open the `test-contributions/test-contributions.md` file.
2. Modify the file adding a new line to it and save the changes.
3. Add your changes to git:

```
git add test-contributions/test-contributions.md
```

4. Create a new commit:

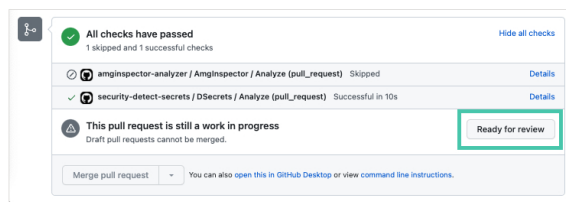
```
git commit -S -m "add your contribution message"
```

5. Create a draft pull request:

```
gh sherpa create-pr  
# Select: y > y
```

This returns the URL of your pull request

6. Open the pull request URL from your web browser.
7. Click on **Ready for review** to update your pull request from Draft to Open status



NOTE As next steps, your pull request would be waiting to be promptly approved by a reviewer to Squash and merged.

SECTION 4

Summary

Conorratulations! You have configured your account and sent your first pull request.

Troubleshooting

If you have any troubles, you can check the [GitHub Ecosystem](#) documentation.

DONE

Development Ecosystem Status All good! See full status page

ECOSYSTEMS

API Village
App Definition & CI/CD
Cloud & Containers
Data & Analytics
Development Frameworks
Development Tools
Mobile
Quality
Security & Compliance
Site Reliability Engineering

COMMUNITY

Explore the company's repos on GitHub
Watch the Chapter videos
[BRAND NAME] tech stack onboarding videos
Sign up for the Technology Newsletter
Cloud Learning for internal users
Cloud Learning for external users

SECTIONS

[BRAND NAME] Open Platform
Platform Overview
The [BRAND NAME] Way of Coding
Path to Production
Check out our Roadmaps
Documentations Catalog
Release Notes
End of Life

BLOG

Share your posts!

Our blog is a place you can all help build and grow with your expertise, your experiences, and your stories.

Do you want to contribute? The door is open! Tell us your idea and we'll work together.

CONTRIBUTE TO OUR BLOG



All rights reserved © [BRAND NAME] 2023

